Project Title -Microservices Deployment on AWS EKS using Docker, Terraform, and Prometheus

Topic: Scalable Microservices Deployment on AWS EKS with Infrastructure as Code (Terraform), Containerization (Docker), and Monitoring (Prometheus + Grafana)

---

2. Project Overview

This project demonstrates the deployment of a microservices-based application on AWS Elastic Kubernetes Service (EKS). The system is containerized using Docker, infrastructure is provisioned using Terraform, and monitoring is implemented using Prometheus and Grafana.

The main objective is to achieve:

- Scalable application deployment
- Automated infrastructure provisioning
- Centralized monitoring and observability
- Cloud-native architecture using Kubernetes

---

3. Technology Stack

- AWS EKS (Kubernetes Cluster)
- Docker (Containerization)
- Terraform (Infrastructure as Code)
- Prometheus (Metrics collection)
- Grafana (Visualization)
- Kubernetes (Orchestration)

---

4. Architecture Explanation

The system follows a microservices architecture:

- Each service is containerized using Docker

- Terraform provisions AWS infrastructure (VPC, EKS, Node Groups)
- Kubernetes manages deployment and scaling
- Prometheus collects cluster metrics
- Grafana visualizes performance dashboards

3. Step by Step Flow—

STEP 1 — PROJECT FOLDER SETUP

Definition

This step creates the complete project directory structure for the CloudCart microservices application.

It organizes:

- Infrastructure code
- Kubernetes manifests
- Frontend service
- Backend microservices

Command

New-Item -ItemType Directory -Force -Path cloudcart-on-eks

Explanation

Creates the main project folder.

Breakdown

- New-Item → Creates new item
- -ItemType Directory → Specifies folder creation
- -Force → Avoids errors if folder exists
- -Path cloudcart-on-eks → Folder name

Command

Set-Location cloudcart-on-eks

Explanation

Moves terminal into the project folder.

Command

New-Item -ItemType Directory -Force -Path infra

Explanation

Creates Terraform infrastructure folder.

Used for:

- EKS code
- VPC code
- IAM code

Command

New-Item -ItemType Directory -Force -Path services

Explanation

Creates folder for all microservices.

Command

New-Item -ItemType Directory -Force -Path services/frontend

Explanation

Creates frontend service folder.

Frontend contains:

- HTML
- CSS
- JS

Command

New-Item -ItemType Directory -Force -Path services/catalog

Explanation

Creates product catalog service folder.

Used to:

- Display products
- Handle product APIs

Command

New-Item -ItemType Directory -Force -Path services/cart

Explanation

Creates cart service folder.

Handles:

- Add cart
- Remove cart
- Cart management

Command

New-Item -ItemType Directory -Force -Path services/order

Explanation

Creates order management service.

Handles:

- Orders
- Checkout
- Purchase processing

Command

New-Item -ItemType Directory -Force -Path services/payment-mock

Explanation

Creates mock payment service.

Used for:

- Demo payments
- Testing payment flow

Command

New-Item -ItemType Directory -Force -Path services/notification

Explanation

Creates notification service.

Handles:

- Email notifications
- Alerts
- Order updates

Command

New-Item -ItemType Directory -Force -Path k8s

Explanation

Creates Kubernetes YAML folder.

Stores:

- Deployments
- Services
- Ingress files

STEP 2 — FILE CREATION

Definition

Creates application source files and configuration files.

Command

New-Item -ItemType File -Force -Path services/frontend/index.html

Explanation

Creates frontend webpage file.

Command

New-Item -ItemType File -Force -Path services/catalog/app.py

Explanation

Creates Python backend application file.

Command

New-Item -ItemType File -Force -Path services/catalog/requirements.txt

Explanation

Stores Python package dependencies.

Example:

flask
requests

Command

New-Item -ItemType File -Force -Path services/catalog/Dockerfile

Explanation

Creates Docker instructions file.

SAME PURPOSE COMMANDS

These commands create:

- app.py
- requirements.txt
- Dockerfile

for:

- cart
- order
- payment-mock
- notification

Commands

```
New-Item -ItemType File -Force -Path services/cart/app.py
New-Item -ItemType File -Force -Path services/cart/requirements.txt
New-Item -ItemType File -Force -Path services/cart/Dockerfile

New-Item -ItemType File -Force -Path services/order/app.py
New-Item -ItemType File -Force -Path services/order/requirements.txt
New-Item -ItemType File -Force -Path services/order/Dockerfile

New-Item -ItemType File -Force -Path services/payment-mock/app.py
New-Item -ItemType File -Force -Path services/payment-mock/requirements.txt
New-Item -ItemType File -Force -Path services/payment-mock/Dockerfile

New-Item -ItemType File -Force -Path services/notification/app.py
New-Item -ItemType File -Force -Path services/notification/requirements.txt
New-Item -ItemType File -Force -Path services/notification/Dockerfile
```

Kubernetes YAML Files

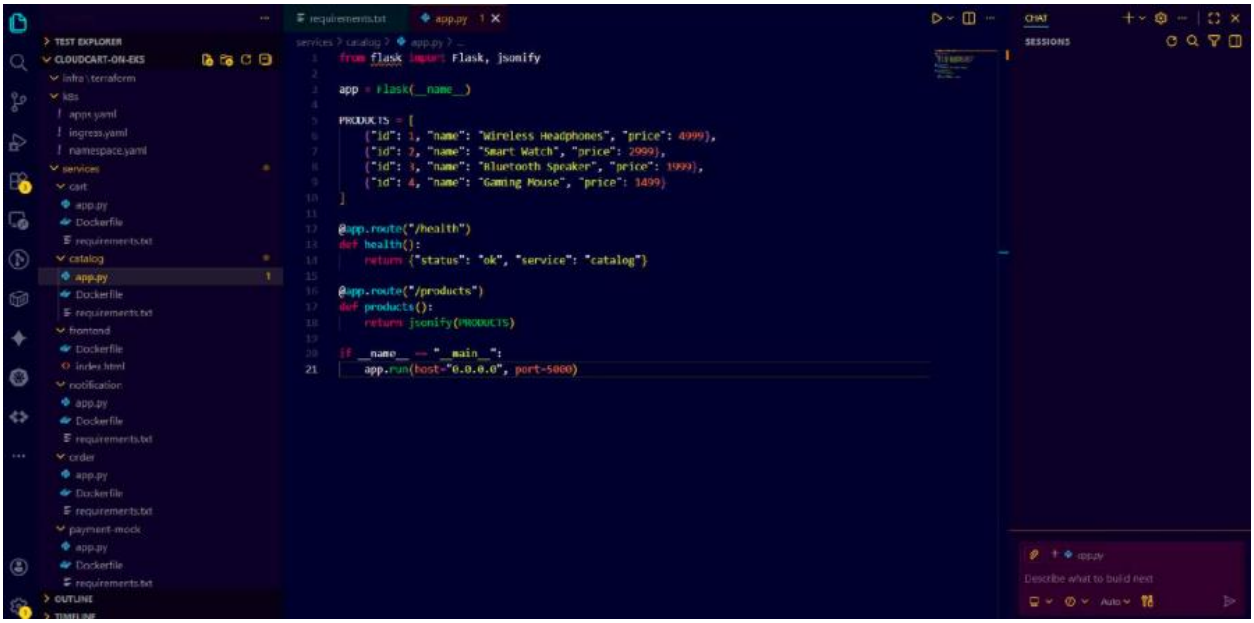
```
New-Item -ItemType File -Force -Path k8s/namespace.yaml
New-Item -ItemType File -Force -Path k8s/apps.yaml
New-Item -ItemType File -Force -Path k8s/ingress.yaml
```

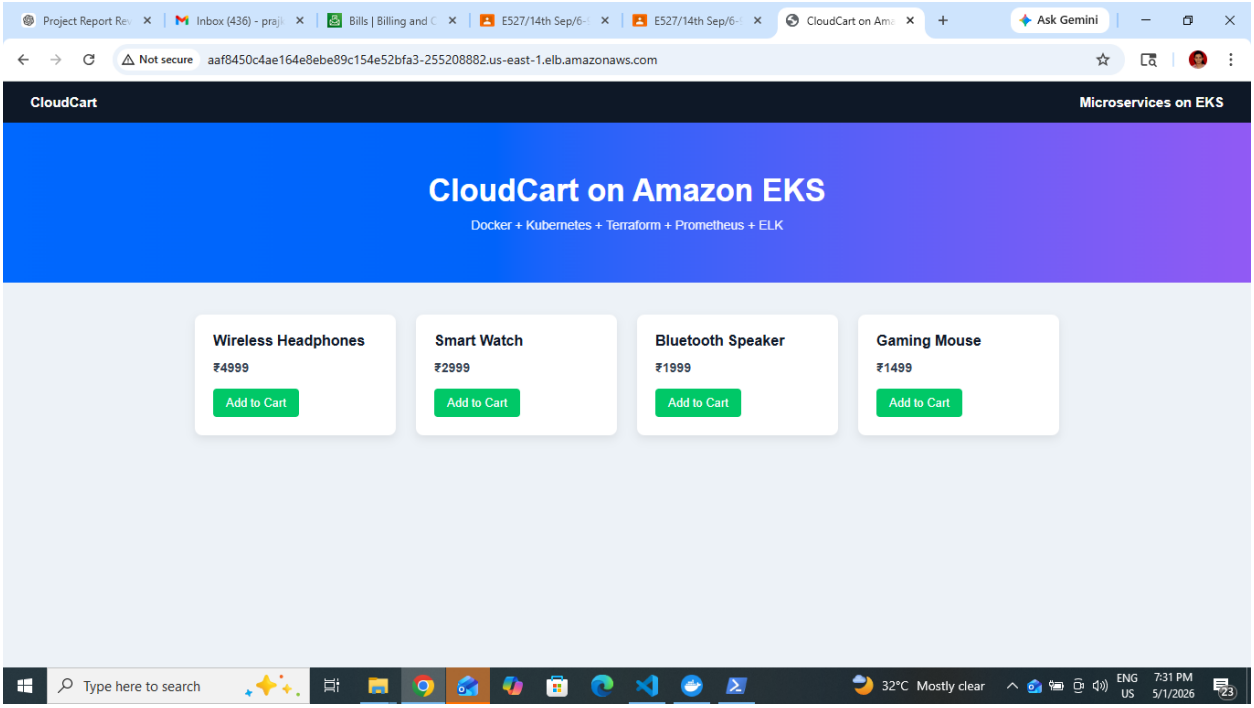
Explanation

Creates Kubernetes manifest files.

Used for:

- Namespace creation
- Deployments
- Services
- Ingress routing





STEP 3 — DOCKER BUILD & RUN

Docker Build

docker build -t catalog:latest .

Explanation

Builds Docker image.

Breakdown

- docker build → Build image
- -t → Tag image
- catalog:latest → Image name
- . → Current directory

Docker Run

docker run -p 5000:5000 catalog:latest

Explanation

Runs Docker container locally.

Port Mapping

- Left side = Local system port
- Right side = Container port

Frontend Docker Build

```
docker build -t frontend:latest .
```

Explanation

Builds frontend Docker image.

Frontend Run

```
docker run -p 8080:80 frontend:latest
```

Explanation

Runs frontend container.

Access:

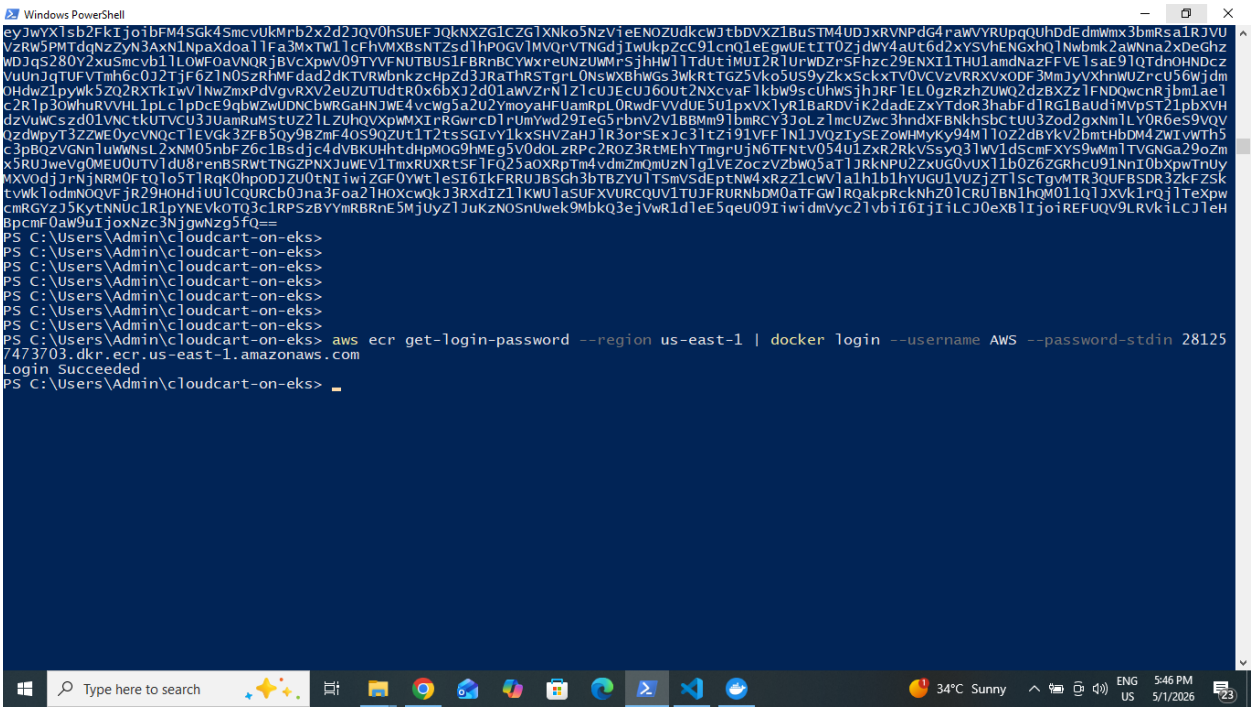
http://localhost:8080

Multiple Image Builds

```
docker build -t frontend:latest .
docker build -t catalog:latest .
docker build -t cart:latest .
docker build -t order:latest .
docker build -t payment-mock:latest .
docker build -t notification:latest .
```

Explanation

Builds all service images.



STEP 4 — AWS SETUP

Configure AWS CLI

aws configure

Explanation

Configures AWS credentials.

Stores:

- Access key
- Secret key
- Region

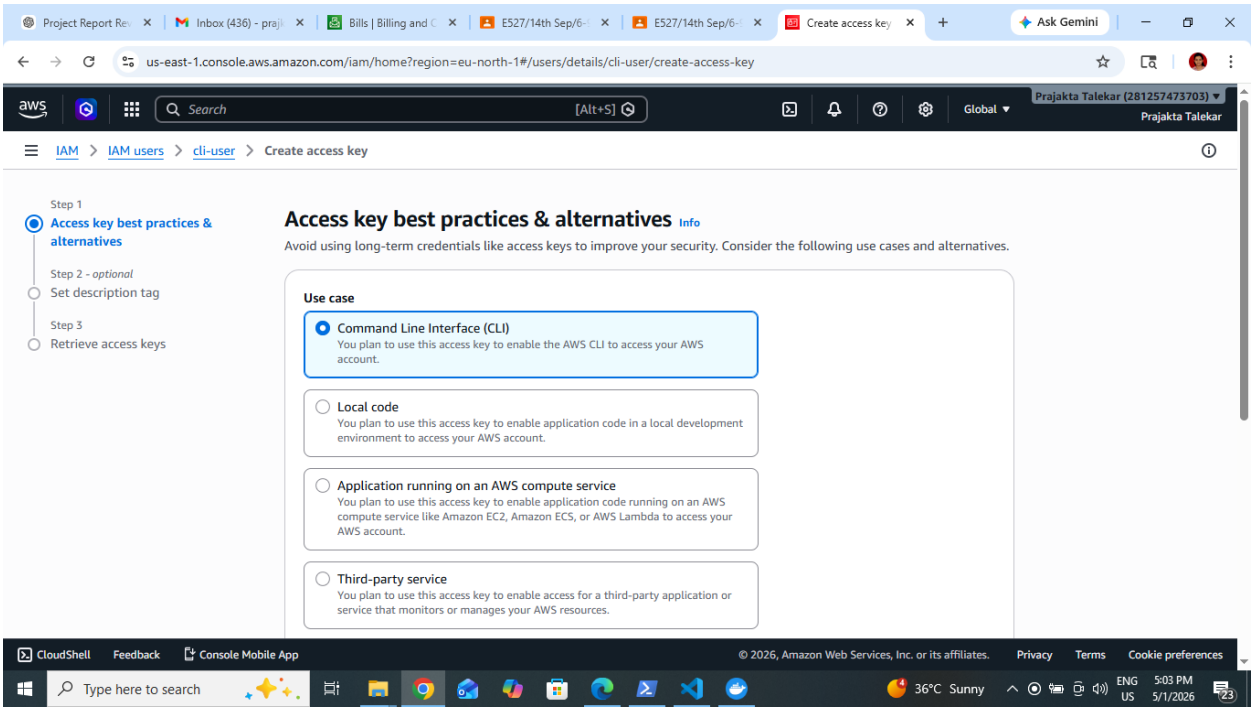
Get AWS Account ID

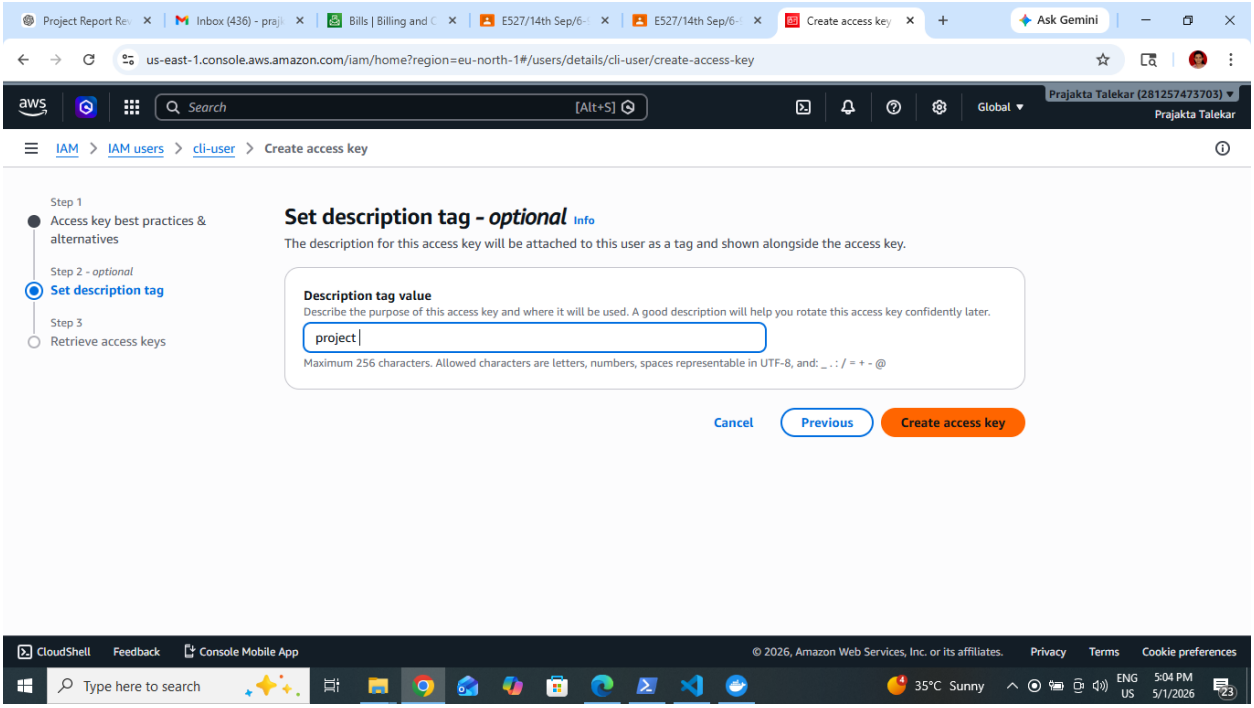
aws sts get-caller-identity --query Account --output text

Explanation

Returns AWS account number.

Used for:

- ECR URLs
- Docker tagging
- 

- 

STEP 5 — CREATE ECR REPOSITORIES

Definition

ECR = Elastic Container Registry

Stores Docker images in AWS.

Commands

```
aws ecr create-repository --repository-name frontend --region us-east-1
aws ecr create-repository --repository-name catalog --region us-east-1
aws ecr create-repository --repository-name cart --region us-east-1
aws ecr create-repository --repository-name order --region us-east-1
aws ecr create-repository --repository-name payment-mock --region us-east-1
aws ecr create-repository --repository-name notification --region us-east-1
```

Explanation

Creates repositories for all services.

Login to ECR

aws ecr get-login-password --region us-east-1

Explanation

Gets temporary Docker authentication password.

Docker Login

docker login --username AWS --password-stdin <account>.dkr.ecr.us-east-1.amazonaws.com

Explanation

Authenticates Docker with ECR.

STEP 6 — DOCKER TAGGING

docker tag frontend:latest <account>.dkr.ecr.us-east-1.amazonaws.com/frontend:latest

Explanation

Adds ECR repository path to image.

All Tag Commands

docker tag catalog:latest <account>.dkr.ecr.us-east-1.amazonaws.com/catalog:latest
docker tag cart:latest <account>.dkr.ecr.us-east-1.amazonaws.com/cart:latest
docker tag order:latest <account>.dkr.ecr.us-east-1.amazonaws.com/order:latest
docker tag payment-mock:latest <account>.dkr.ecr.us-east-1.amazonaws.com/payment-mock:latest
docker tag notification:latest <account>.dkr.ecr.us-east-1.amazonaws.com/notification:latest

---

STEP 7 — PUSH IMAGES TO ECR

docker push <account>.dkr.ecr.us-east-1.amazonaws.com/frontend:latest

Explanation

Uploads image to AWS ECR.

All Push Commands

```
docker push <account>.dkr.ecr.us-east-1.amazonaws.com/catalog:latest
docker push <account>.dkr.ecr.us-east-1.amazonaws.com/cart:latest
docker push <account>.dkr.ecr.us-east-1.amazonaws.com/order:latest
docker push <account>.dkr.ecr.us-east-1.amazonaws.com/payment-mock:latest
docker push <account>.dkr.ecr.us-east-1.amazonaws.com/notification:latest
```

STEP 8 — TERRAFORM / EKS

Check Terraform Version

```
terraform version
```

Explanation

Checks installed Terraform version.

Initialize Terraform

```
terraform init
```

Explanation

Downloads:

- AWS provider
- Terraform plugins

Terraform Plan

```
terraform plan
```

Explanation

Shows infrastructure preview.

Terraform Apply

terraform apply -auto-approve

Explanation

Creates:

- EKS cluster
- VPC
- Nodes

Connect kubectl to EKS

aws eks --region us-east-1 update-kubeconfig --name cloudcart-eks

Explanation

Connects kubectl with EKS cluster.

Check Nodes

kubectl get nodes

Explanation

Shows Kubernetes worker nodes.

STEP 9 — KUBERNETES DEPLOYMENT

kubectl apply -f k8s.yaml

Explanation

Deploys applications to Kubernetes.

Get Pods

kubectl get pods -n cloudcart

Explanation

Shows running containers.

Get Services

kubectl get svc -n cloudcart

Explanation

Shows networking services.

Get Ingress

kubectl get ingress -n cloudcart

Explanation

Shows external routing information.

STEP 10 — INSTALL INGRESS CONTROLLER

kubectl apply -f https://raw.githubusercontent.com/kubernetes/ingress-nginx/main/deploy/static/provider/cloud/deploy.yaml

Explanation

Installs NGINX ingress controller.

Check Ingress Pods

kubectl get pods -n ingress-nginx

Explanation

Checks ingress controller status.

Check Ingress Services

kubectl get svc -n ingress-nginx

Explanation

Shows LoadBalancer external URL.

Describe Ingress

kubectl describe ingress cloudcart-ingress -n cloudcart

Explanation

Displays detailed ingress configuration.

Logs

kubectl logs -n cloudcart deployment/catalog --all-pods=true

Explanation

Checks application logs for debugging.

Create Monitoring Namespace

kubectl create namespace monitoring

Explanation

Creates namespace for monitoring tools.

```
Windows PowerShell
PS C:\Users\Admin\cloudcart-on-eks\infra> kubectl get pods -n ingress-nginx
NAME                                        READY   STATUS             RESTARTS   AGE
ingress-nginx-admission-create-vc4pm        0/1     Completed          0          5s
ingress-nginx-admission-patch-fwrqk         0/1     Completed          0          4s
ingress-nginx-controller-85f5f9dfcd-2kbt7   0/1     ContainerCreating  0          6s
PS C:\Users\Admin\cloudcart-on-eks\infra> kubectl get svc -n ingress-nginx
NAME                                 TYPE           CLUSTER-IP      EXTERNAL-IP
        PORT(S)                      AGE
ingress-nginx-controller             LoadBalancer   172.20.26.90    aaf8450c4ae164e8ebe89c154e52bfa3-255208882.us-east-1.elb.amazonaws
.com   80:32368/TCP,443:30583/TCP   11s
ingress-nginx-controller-admission   ClusterIP      172.20.105.19   <none>
        443/TCP                      10s
PS C:\Users\Admin\cloudcart-on-eks\infra>
PS C:\Users\Admin\cloudcart-on-eks\infra> kubectl describe ingress cloudcart-ingress -n cloudcart
Error from server (NotFound): namespaces "cloudcart" not found
PS C:\Users\Admin\cloudcart-on-eks\infra>
PS C:\Users\Admin\cloudcart-on-eks\infra> kubectl logs -n cloudcart deployment/catalog --all-pods=true
error: error from server (NotFound): namespaces "cloudcart" not found in namespace "cloudcart"
PS C:\Users\Admin\cloudcart-on-eks\infra>
PS C:\Users\Admin\cloudcart-on-eks\infra> kubectl create namespace monitoring
namespace/monitoring created
PS C:\Users\Admin\cloudcart-on-eks\infra>
PS C:\Users\Admin\cloudcart-on-eks\infra>
Type here to search
33°C Clear
ENG US
7:09 PM 5/1/2026
```

STEP 11 — GRAFANA / PROMETHEUS

Add Helm Repo

helm repo add prometheus-community https://prometheus-community.github.io/helm-charts

Explanation

Adds monitoring chart repository.

Update Helm

helm repo update

Explanation

Downloads latest chart data.

Install Monitoring Stack

helm install monitoring prometheus-community/kube-prometheus-stack -n monitoring

Explanation

Installs:

- Prometheus
- Grafana

Monitoring Pods

kubectl get pods -n monitoring

Explanation

Shows monitoring containers.

Monitoring Services

kubectl get svc -n monitoring

Explanation

Shows monitoring networking services.

Port Forward Grafana

kubectl port-forward -n monitoring svc/monitoring-grafana 3000:80

Explanation

Opens Grafana dashboard locally.

Access:

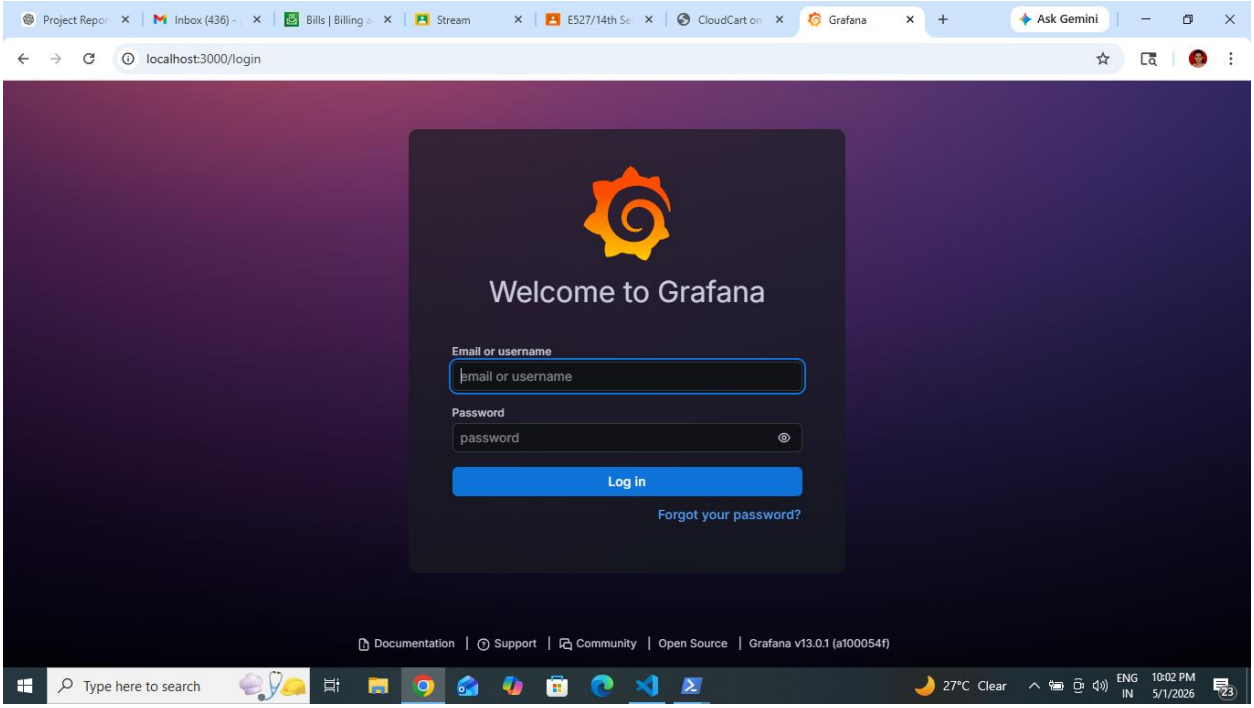
http://localhost:3000

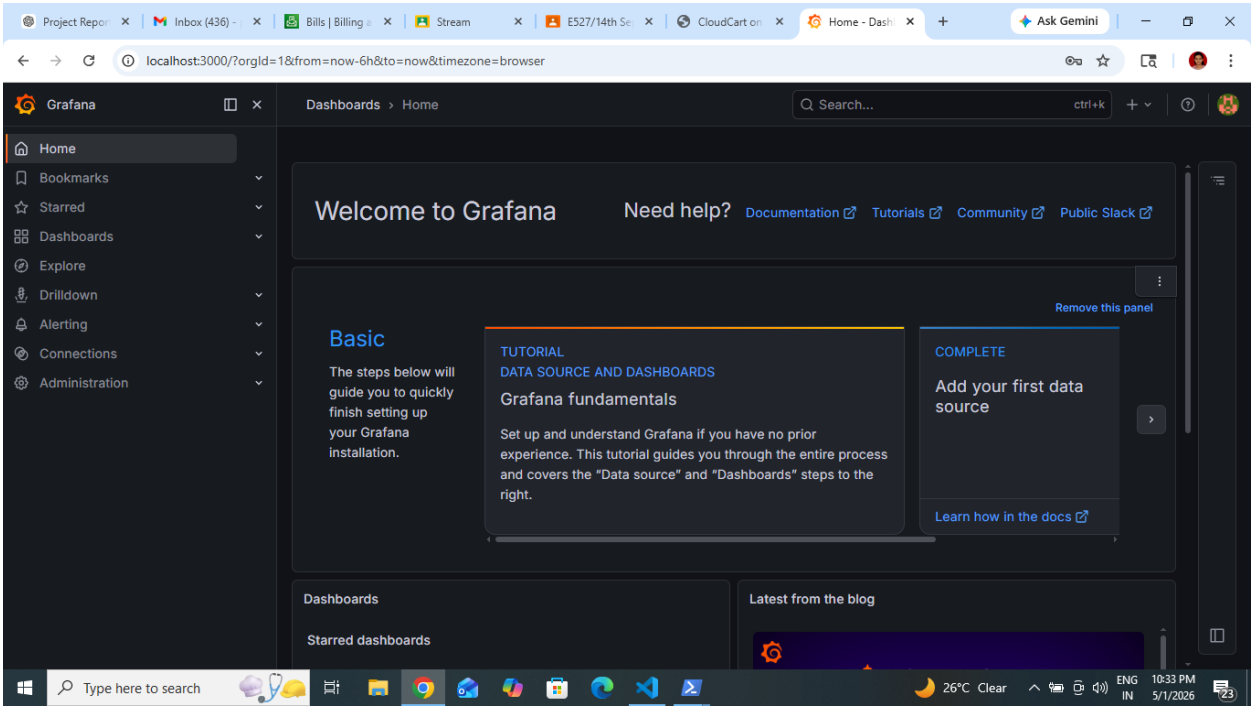
Get Grafana Password

kubectl get secret -n monitoring monitoring-grafana -o jsonpath="{.data.admin-password}"

Explanation

Gets Grafana admin password.



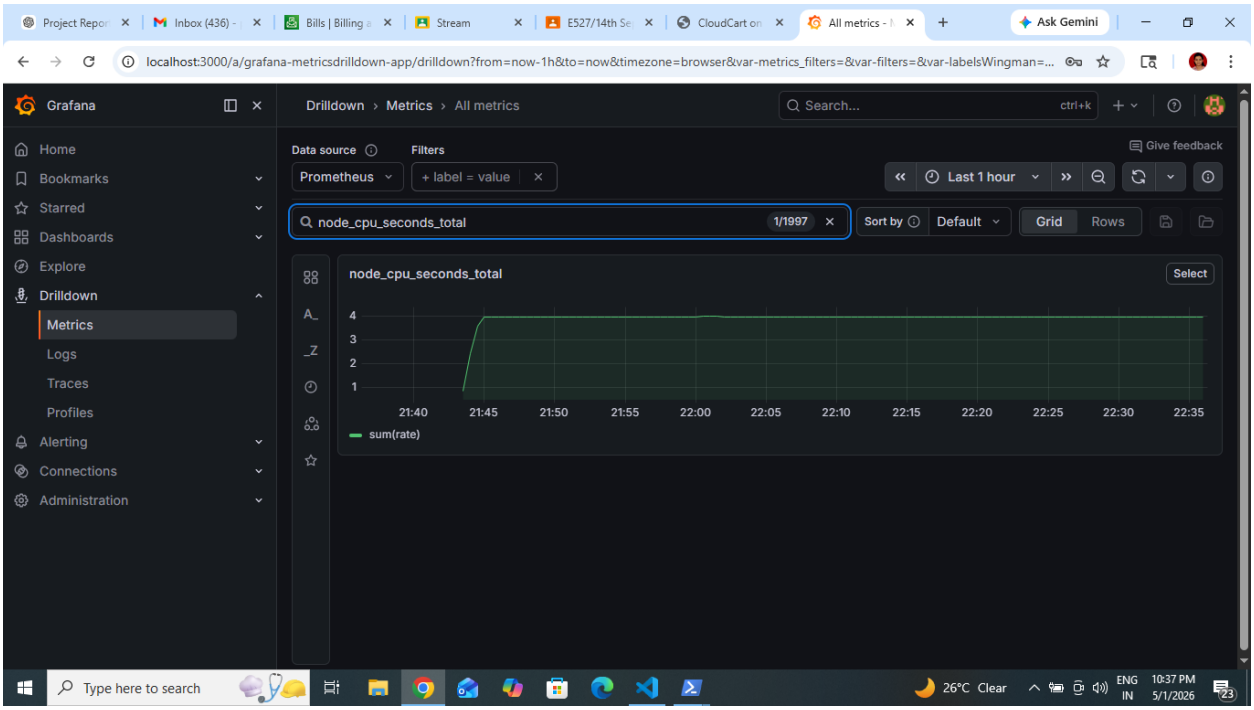


"Grafana was integrated with Prometheus using Helm charts. Prometheus collects metrics from Kubernetes nodes and pods, while Grafana visualizes those metrics using dashboards. We monitored CPU usage, pod health, and cluster availability in real time."

Node_cpu_seconds_total- node_cpu_seconds_total is a Prometheus metric used to monitor **CPU usage of Kubernetes/EKS worker nodes**.

It tells us:

- How much CPU time the node is spending
- Whether CPU is idle or busy
- System performance of the server/node
- CPU utilization trends over time



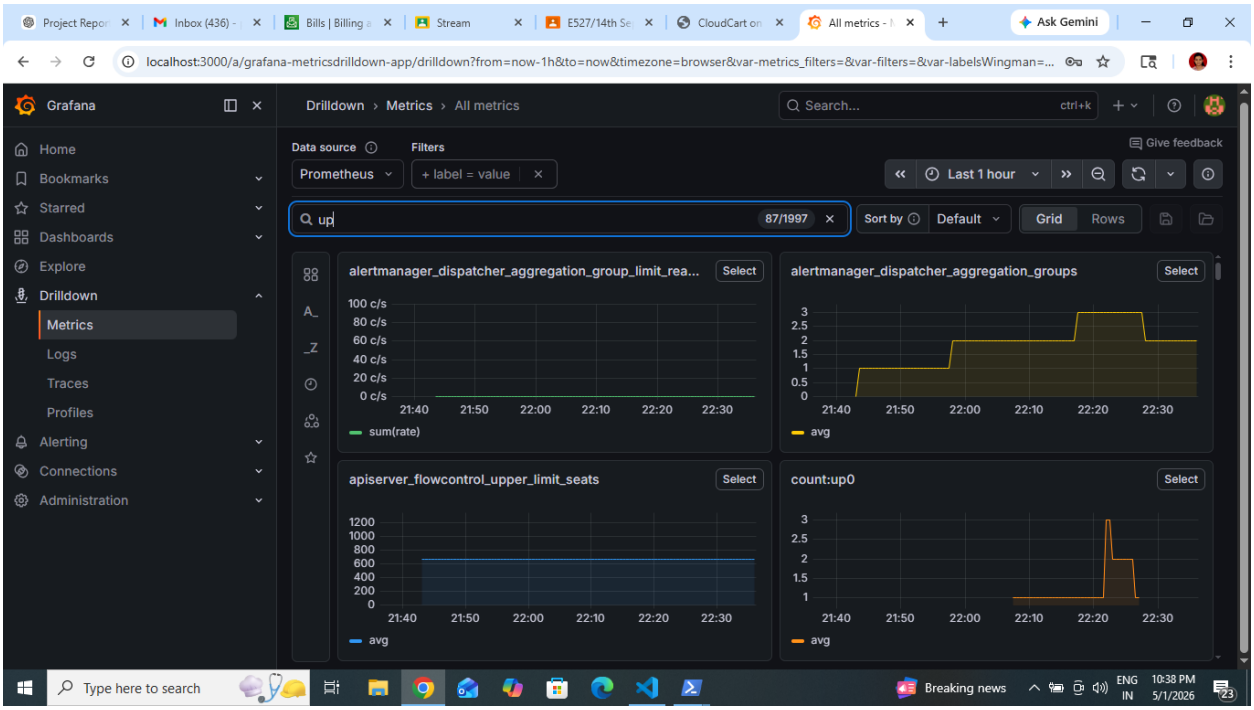
Up- The Prometheus metric:

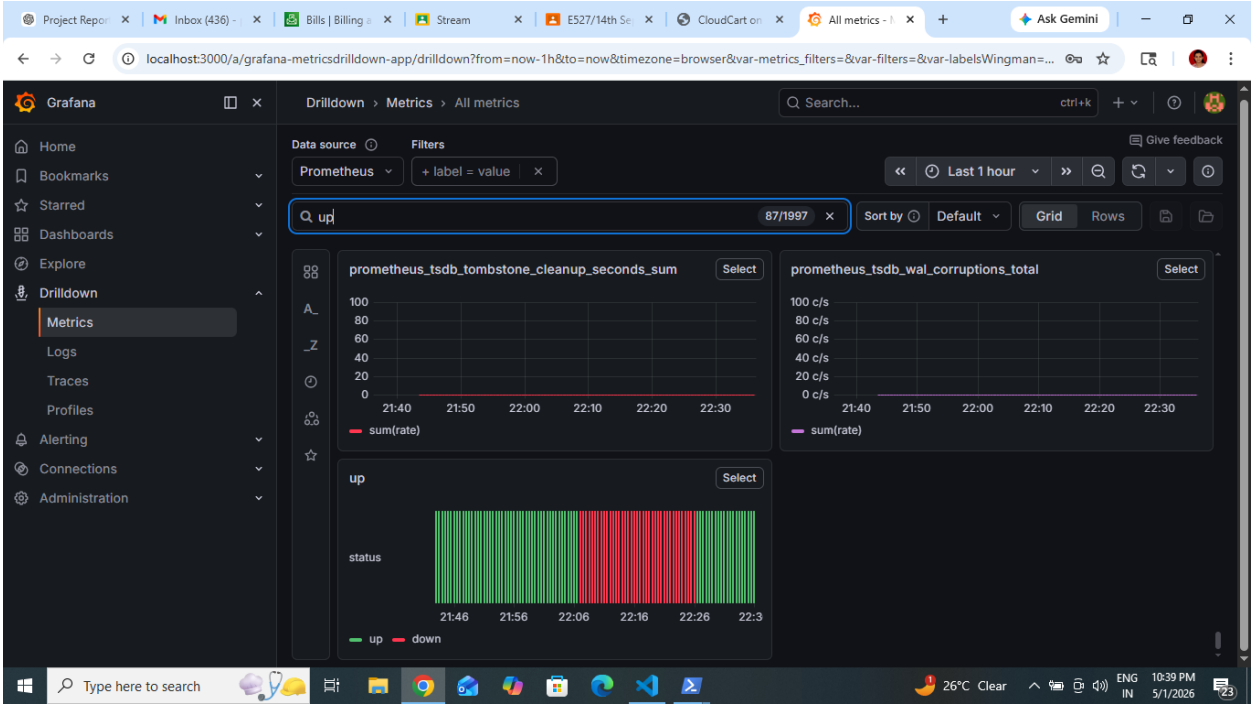
is one of the MOST important monitoring metrics.

It tells whether a target/service is:

- Running
- Reachable
- Healthy
- Successfully monitored by Prometheus

- When you searched: UP
    - Grafana showed green bars.
    - That means:
    - Prometheus successfully connected
      Kubernetes targets reachable
      Monitoring active
      Services healthy

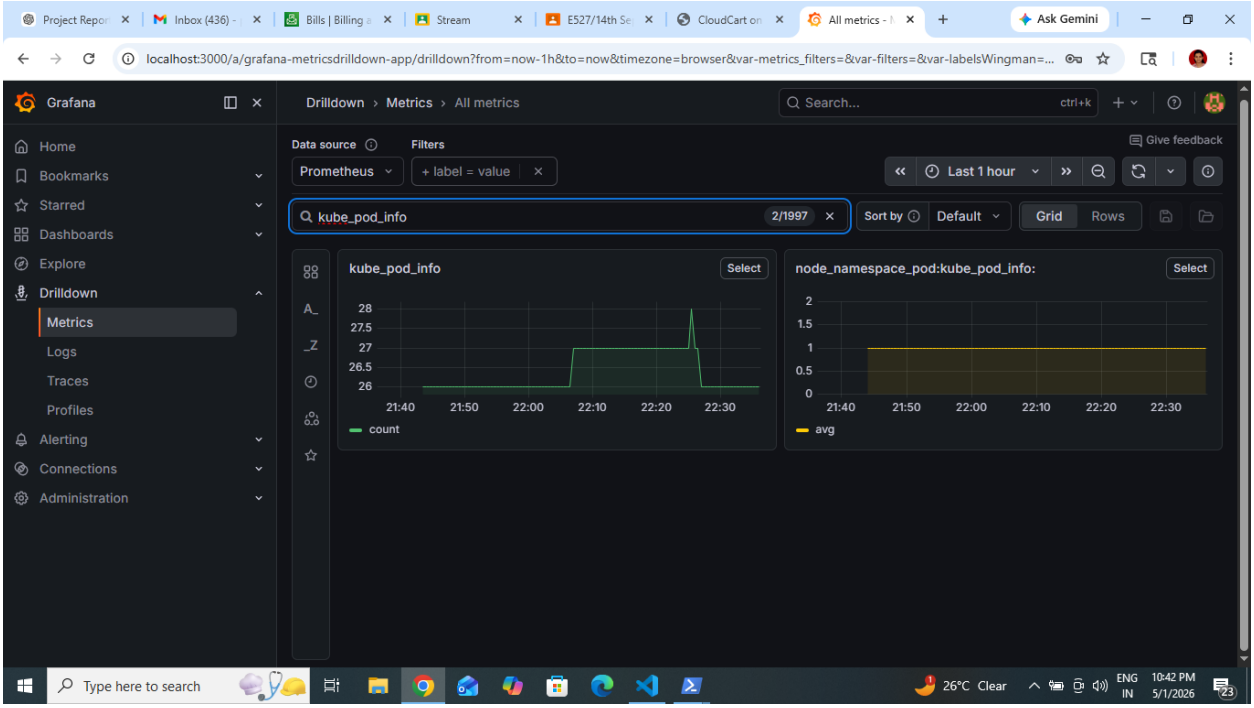




Kube_pod_info---- is a Prometheus metric used to get information about Kubernetes pods running inside the cluster.

It provides metadata about pods such as:

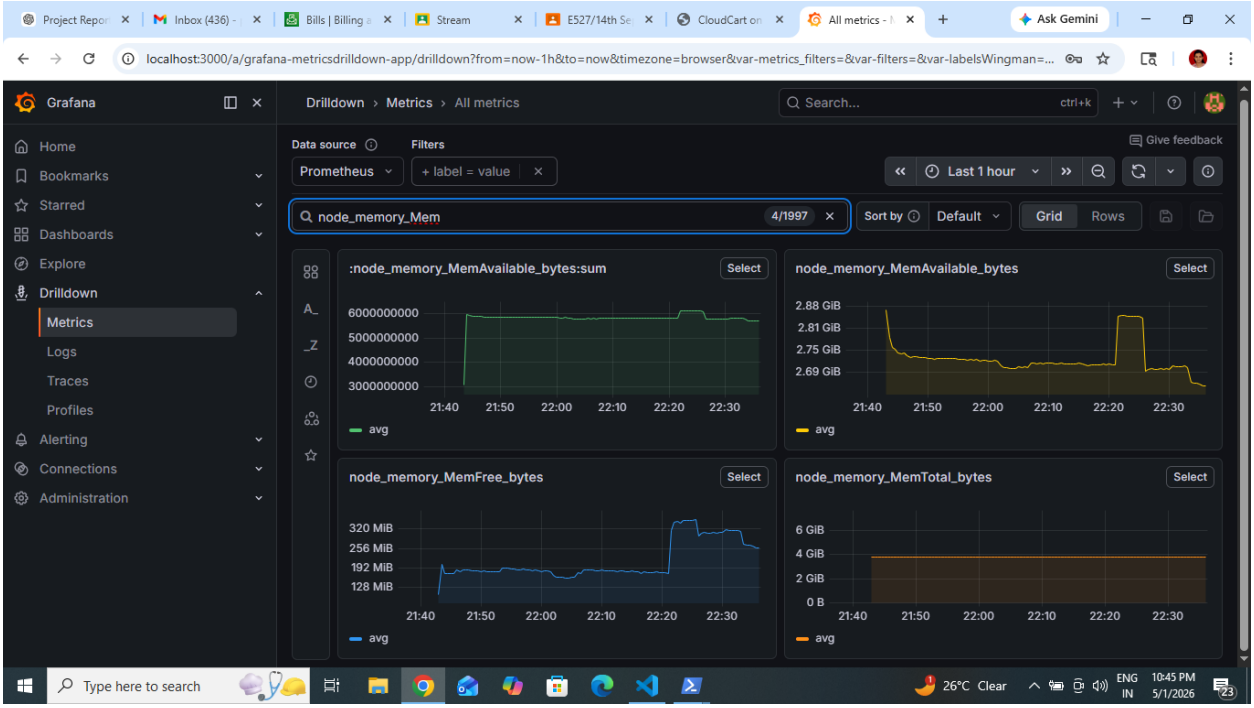
- Pod name
- Namespace
- Node name
- Pod UID
- Cluster pod details



node_memory_MemAvailable----_bytes is a Prometheus metric used to monitor **available RAM (memory)** on Kubernetes/EKS worker nodes.

It tells us:

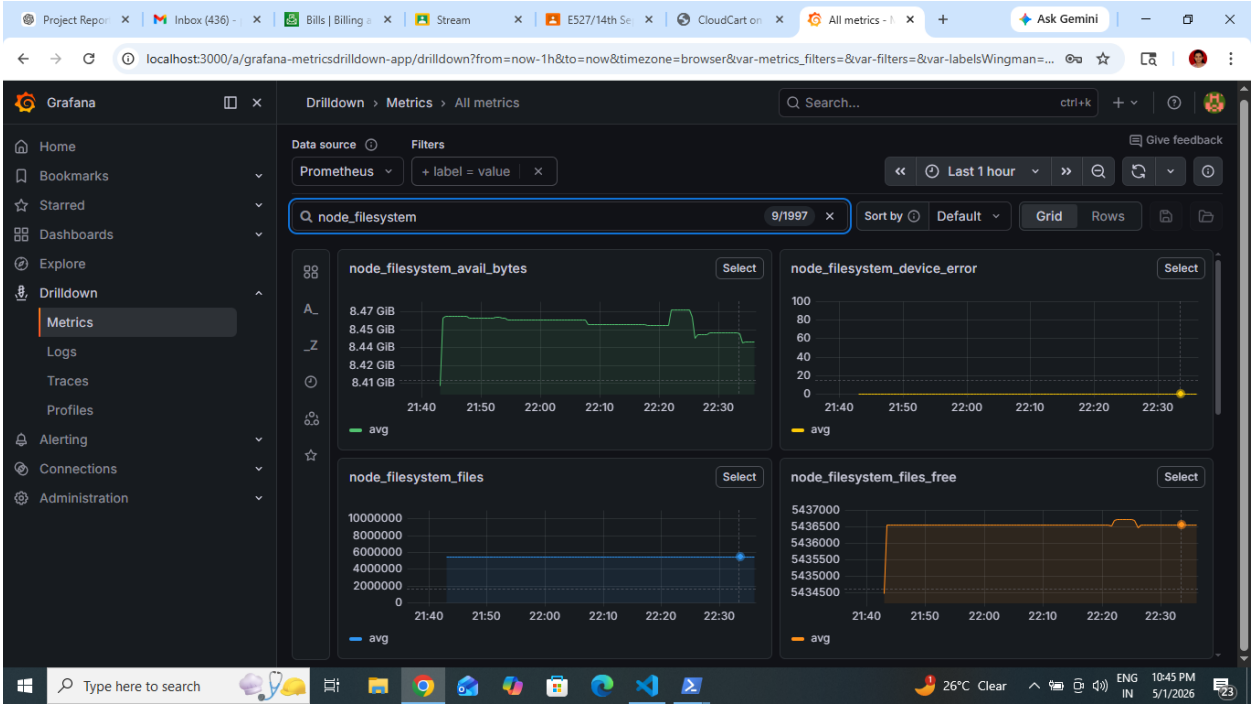
- How much memory is free
- Current RAM availability
- System memory health
- Whether nodes are under memory pressure



Node_filesystem-- metrics are Prometheus metrics used to monitor the **disk storage/filesystem usage** of Kubernetes/EKS worker nodes.

These metrics help check:

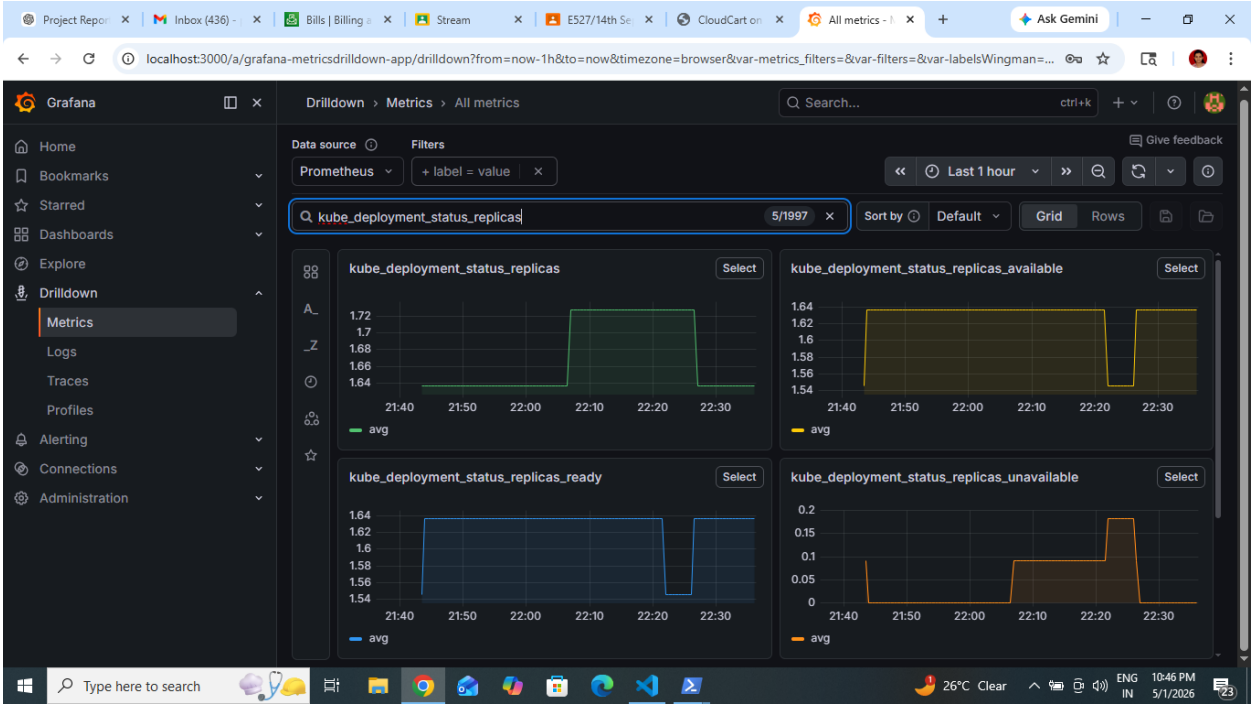
- Disk space usage
- Free storage available
- Filesystem health
- Storage performance



Kube_deployment_status_replicas---- is a Prometheus metric used to monitor the **number of replicas (pods)** running for a Kubernetes deployment.

It tells us:

- How many pod replicas are running
- Deployment availability
- Scaling status



kubectl get ns

before cleanup to:

Verify all namespaces created successfully
Check application namespace exists
Confirm monitoring namespace exists
Ensure ingress namespace exists

It acts like a final verification step.

Windows PowerShell

```
PS C:\Users\Admin> kubectl port-forward -n monitoring svc/monitoring-grafana 3000:80^C
PS C:\Users\Admin> [System.Text.Encoding]::UTF8.GetString([System.Convert]::FromBase64String((kubectl get secret -n monitoring monitor
ing-grafana -o jsonpath="{.data.admin-password}")))
FrmvvnlgupzOzNUODdlQazMhSk4kxrz7rWWNFMOa
PS C:\Users\Admin> kubectl get ns
NAME              STATUS   AGE
cloudcart         Active   3h32m
default           Active   4h12m
ingress-nginx     Active   3h41m
kube-node-lease   Active   4h12m
kube-public       Active   4h12m
kube-system       Active   4h12m
monitoring        Active   3h39m
PS C:\Users\Admin>
```

Type here to search

26°C Clear

ENG IN 10:48 PM 5/1/2026